

ProCode

COLLEGE DSA & CODING PRACTICE PLATFORM

A curriculum-aligned coding platform built for first-year students to practice syllabus DSA, compete on class leaderboards, and seamlessly graduate to LeetCode.

TECHNICAL ARCHITECTURE | HOSTING RUNBOOK | STUDENT USER MANUAL

Document Overview & Metadata

Property	Platform Specification
Platform Name	ProCode (College DSA Practice Platform)
Target Audience	First-Year Engineering Students, Faculty & Administrators
Core Architecture	MERN Stack (MongoDB, Express, React, Node) + Judge0 Execution Engine
Gamification Model	Section/Class Leaderboards (Daily, Weekly, Career) + Bronze-to-Conqueror Tiers
Exam Preparation	Unit-wise Problem Ladder + Integrated College PYQ Library + Concept Notes
Bridge Mechanism	Automatic LeetCode Equivalent Linkage upon Problem Solve
Deployment Targets	Frontend: Vercel / Netlify Backend: Render / VPS Database: Atlas
Document Version	1.0.0 (Production Master Release)

EXECUTIVE SUMMARY

Generic platforms like LeetCode or GeeksforGeeks fail first-year college beginners because they assume prior knowledge and disconnect from college exam patterns. ProCode solves this by mapping problems directly to semester units and past papers, while offering batch-level rivalry and a direct graduation bridge to LeetCode.

1. How ProCode Was Built & Architectural Blueprint

ProCode is engineered as a decoupled, resilient, mobile-first client-server platform. It isolates the high-risk, computationally heavy task of code execution from general web browsing, ensuring that even under severe student concurrency spikes (e.g. during lab exams or batch contests), the platform remains responsive and never crashes.

1.1 High-Level Component Flow

The system is structured across four primary cooperative layers:

Component	Role & Implementation Details
Client SPA (React + Vite)	Responsive web interface with dark navy theme, interactive multi-language code editor, syllabus ladders, and live batch podium.
API Gateway (Express.js)	Stateless REST server managing authentication (JWT), rate-limiting safeguards, anti-cheat reporting, and gamification calculations.
Execution Bridge (Judge0 / Local)	Sandboxed code execution proxy supporting C, C++, Java, and Python with CPU time (2.0s) and memory caps (128MB). Includes offline fallback evaluator.
Persistence (MongoDB / Local Store)	Dual-mode database storing student profiles, problem test cases, PYQs, concept notes, and immutable submission audit logs.

1.2 Gamification & Point Integrity Algorithm

To eliminate "point-farming" where students repeatedly re-submit the same solution, points are awarded exactly once per unique problem. Rank tiers only move upward (no demotion), preserving beginner confidence:

Difficulty Level	Points Awarded	Criteria & Constraints
Easy	10 Points	Basic 1D array/string iterations, single loop O(N) operations.
Medium	25 Points	Two pointers, recursion trees, stack applications, sorting.
Hard	50 Points	Topic graduation problems, optimal two-pointer math (e.g. Water Container).

Rank Tiers: Bronze (0–79 pts) -> Silver (80–299 pts) -> Gold (300–699 pts) -> Platinum (700–1499 pts) -> Diamond (1500–2999 pts) -> Conqueror (3000+ pts).

1.3 Reliability, Crash Resilience & Defensive Design

In accordance with the build roadmap, multiple fail-safes are hardcoded:

- Express Global Error Middleware: All uncaught exceptions return structured HTTP 500 JSON without terminating the Node.js process.
- React Error Boundary: Catches runtime UI rendering exceptions and renders a helpful recovery button instead of a blank white screen.

- Maintenance Interceptor: Toggled via `MAINTENANCE_MODE=true` in `.env`; immediately serves a friendly "Back in 10 minutes" screen to prevent users hitting partial states during upgrades.
- Rate Limiting: IP-based sliding window restricts aggressive submission bursts and protects authentication endpoints from brute force.

2. Complete Technology Stack Breakdown

The technologies powering ProCode were deliberately chosen for rapid responsiveness, zero-friction developer setup, and effortless cloud scaling:

Tier	Technology Choice	Technical Rationale & Advantage
Frontend Core	React 19 + Vite 8	Instant HMR, blazing-fast JSX bundling, and lightweight modern rendering architecture.
UI & Styling	Tailwind CSS v4	Utility-first styling with zero CSS bloat; built-in sm:, md:, lg: responsive breakpoints for mobile devices.
Icons & Effects	Lucide React + Canvas Confetti	Clean SVG icons and celebratory particle physics upon problem solve.
Backend Server	Node.js v24 + Express 4	Asynchronous event-driven I/O ideal for handling concurrent student submission streams.
Security & Auth	JSON Web Tokens (JWT) + Bcrypt	Stateless cryptographic token authentication and salted one-way password hashing.
Traffic Defense	express-rate-limit	Protects against DDOS, submission spamming, and API enumeration.
Database Layer	Dual-Mode (Mongoose + JSON)	Full MongoDB Atlas support in production + zero-setup local persistent fallback for offline running.
Execution Sandboxing	Judge0 Engine (API / Docker)	Multi-language isolated Linux container runner for C, C++, Java, and Python with hard resource quotas.

2.1 Why Dual-Mode Database Architecture?

During local evaluation or in environments without an active MongoDB daemon, traditional MERN apps crash on startup. ProCode incorporates a transparent `JsonCollection` adapter in `services/db.js`. If `MONGODB_URI` is provided in `.env`, it seamlessly binds to MongoDB Atlas; otherwise, it synchronizes atomically with `local.store.json`, making it completely zero-friction for examiners or student demonstrations.

2.2 Sandboxed Execution Safety

Student-submitted code is inherently untrusted — infinite loops (`while(1)`), excessive heap allocation, or file system access attempts are common. By delegating compilation to Judge0 containers with 2-second CPU limits and 128MB memory caps, student code cannot affect the host Node.js application.

3. Hosting Instructions for Platform Owner (You)

Follow these step-by-step instructions to host and operate ProCode in production for your college or cohort.

3.1 Step 1: Database Setup (MongoDB Atlas — Free Tier)

1. Create a free account on mongodb.com/cloud/atlas.
2. Deploy a free M0 Shared Cluster in your nearest AWS region (e.g. Mumbai / Singapore).
3. In "Database Access", create a user (e.g., `procode_admin`) and secure password.
4. In "Network Access", add IP Address `0.0.0.0/0` (Allow access from anywhere).
5. Under "Connect" -> "Drivers", copy your MongoDB connection URI:

```
mongodb+srv://procode_admin:<password>@cluster0.procode.mongodb.net/procode_db?
retryWrites=true&w=majority
```

3.2 Step 2: Code Execution Setup (Judge0)

Choose one of two options:

- Option A (Fastest): Subscribe to Judge0 CE on RapidAPI (rapidapi.com/hermanzdosilovic/api/judge0-ce). Copy your RapidAPI Key.
- Option B (Self-Hosted): Deploy Judge0 CE using Docker on a \$5/month DigitalOcean droplet:

```
curl -s https://raw.githubusercontent.com/judge0/judge0/master/docker-compose.yml -o docker-
compose.yml
docker-compose up -d
```

3.3 Step 3: Backend Deployment (Render.com / VPS)

1. Connect your GitHub repository to Render.com and create a new "Web Service".
2. Set Root Directory to "server", Runtime to "Node", Build Command to "npm install", and Start Command to "node index.js".
3. Configure Environment Variables in the Render dashboard:

Variable Name	Recommended Production Value
PORT	5000 (or Render default)
MONGODB_URI	mongodb+srv://procode_admin:... (from Step 1)
JWT_SECRET	your_long_random_cryptographic_secret_string
JUDGE0_API_URL	https://judge0-ce.p.rapidapi.com (or your Docker host)
JUDGE0_API_KEY	your_rapidapi_judge0_key
MAINTENANCE_MODE	false (set to true only during updates)

4. Health Check Verification: Test `https://your-backend.onrender.com/api/health`. It must return status: "ok".

3.4 Step 4: Frontend Deployment (Vercel / Netlify)

1. In Vercel, import your Git repo and choose the "client" folder as the root.
2. Build Command: "npm run build" | Output Directory: "dist".
3. Deploy! Vercel provides free SSL, automatic global CDN, and DDoS protection.

3.5. Handling 1,000 Concurrent Students & Maintenance

When hosting internal college exams or competitive programming contests with up to 1,000 students online at once, apply these production guidelines (Roadmap Page 7, 9 & 10):

A. Isolating Browsing from Execution Traffic

Browsing problem descriptions, reading concept notes, and checking leaderboards consumes almost zero CPU on your server. Code submission is the bottleneck. By utilizing Judge0 queueing, submissions are processed in worker threads without exhausting Express socket pools.

B. PM2 Process Clustering (on VPS / Ubuntu)

If self-hosting the backend on an Ubuntu VPS, run Node.js in cluster mode across all CPU cores:

```
npm install -g pm2
pm2 start index.js -i max --name "procode-api"
pm2 startup
pm2 save
```

C. Activating Planned Maintenance Mode

Before updating syllabus problem sets or modifying database schemas:

1. Set `MAINTENANCE_MODE=true` in your server environment variables.
2. Redeploy or restart the backend. All incoming API calls immediately receive a friendly 503 response stating: "ProCode is undergoing scheduled maintenance. Back in 10 minutes."
3. Perform your updates safely, then set `MAINTENANCE_MODE=false` to restore student access.

UPTIME MONITORING TIP

Configure free uptime monitoring with UptimeRobot (uptimerobot.com) pointed at `/api/health` every 3 minutes. If downtime occurs, you will receive immediate Telegram/Email alerts.

4. User Instructions for Students (How to Use ProCode)

Welcome to ProCode! This platform was built specifically to help you ace your internal college exams, master DSA fundamentals, and graduate confidently to LeetCode.

4.1 Creating Your Account & Joining Your Batch

1. Click "Join / Sign In" at the top right of the navigation bar.
2. Switch to the "Register Student" tab.
3. Enter your Full Name, College Email ID, and Password.
4. Select your College Batch / Section (e.g. CSE-1A, IT-1B, ECE-1A). This links you directly to your class leaderboard!
5. Choose an optional Public Alias if you prefer displaying a handle instead of your legal name.

4.2 Navigating the Problem Ladder

The problem ladder is structured strictly in college syllabus sequence:

- Unit 1: Arrays (Linear data structures, two-pointer swaps, memory formulas)
- Unit 2: Strings (Palindrome checking, pattern matching)
- Unit 3: Recursion (Tower of Hanoi, Fibonacci trees, recurrence relations)
- Unit 4: Searching (Binary search variations, $O(\log N)$ proofs)
- Unit 5 & 6: Stacks & Queues (Balanced parentheses, infix-to-postfix expressions)

Each problem displays its official College Exam Citation Tag (e.g., "Mid-Term Oct 2024, 5 Marks") so you know exactly which university exam tested this concept.

4.3 Coding in the Studio & Running Tests

1. Select your preferred programming language: C, C++, Java, or Python from the top bar. ProCode automatically provides an exam-ready boilerplate template.
2. Write your algorithm in the code editor.
3. Click "Run Cases" to test your solution against visible Sample Test Cases.
4. Use the "Custom Input" tab to supply your own test inputs for edge-case debugging.
5. When confident, click "Submit Code" to evaluate your solution against the protected Hidden Test Cases.

4.4 Earning Points & Climbing Rank Tiers

When all hidden test cases pass, you will receive an "Accepted" verdict with celebratory confetti:

- Easy Problems: +10 Points | Medium Problems: +25 Points | Hard Problems: +50 Points

- Points are awarded ONCE per problem — you cannot farm points by re-solving the same problem.
- Rank Tiers only move UPWARD (no demotion): Bronze -> Silver -> Gold -> Platinum -> Diamond -> Conqueror.

4.5 The LeetCode Graduation Bridge

Every topic ends with a starred Graduation Problem. Once solved, ProCode unlocks direct links to the equivalent LeetCode problem along with 2 recommended follow-ups (Easy -> Medium) to help you transition to tech placement interviews.

4.6 Accessing PYQ Exam Papers & Concept Notes

- PYQ Papers: Visit the "PYQ Papers" tab to search and download past semester exam papers, mid-term question sets, and practical lab viva prompts.
- Exam Notes: Visit "Exam Notes" for concise, high-yield formula summaries and derivations that professors love to test in 10-mark theory questions.

